RECONCILEPRO ENGINE ARCHITECTURE | TECHNICAL SPECIFICATION

reconcile.py - Core GST Reconciliation Engine

A mathematical, algorithmic, and statutory deep-dive into automated reconciliation, schema normalization, safe duplicate resolution, and exception classification.

TARGET FILE	MODULE DIRECTORY	EXECUTION ENVIRONMENT	STATUTORY FRAMEWORK
reconcile.py	backend/reconciliation	Python 3.10+ / Pandas / FastAPI	CGST Act (Sec 16(2)(aa), Sec 50)

1. Executive Summary & Statutory Context

Under the Indian Goods and Services Tax (GST) framework, businesses must reconcile their internal accounting records with statutory portal filings. Mismatches lead to substantial interest liabilities under Section 50 of the CGST Act and disallowance of **Input Tax Credit (ITC)** under Section 16(2)(aa).

Outward Reconciliation (Sales)	Inward Reconciliation (Purchases)
Source 1: Sales Register (Books) **Source 2:** GSTR-1 Portal Return **Primary Goal:** Detect unfiled outward supplies, rate variances, and invoice number typos to avoid tax liability interest.	**Source 1:** Purchase Register (Inward Ledger) **Source 2:** GSTR-2A / GSTR-2B Returns **Primary Goal:** Verify vendor return filing, eliminate unfiled vendor claims, and safeguard Input Tax Credit.

2. Mathematical Set Partitioning Framework

Rather than utilizing quadratic comparisons (O(N * M) nested iterations), the engine projects normalized invoice identifier strings into Python hash sets. This establishes a 3-way disjoint set partition operating in optimal **O(N + M) linear time**:

Universe of Invoice Identifiers: U
- $\mathbf{S_{my}}$ = { inv in Books | inv != '' } - $\mathbf{S_{gst}}$ = { inv in Portal | inv != '' }
1. **Common Intersection:** $S_{common} = S_{my}$ (intersect) S_{gst} -> Evaluated for 1:1 Value Match, Value Mismatch, or Duplicate Collisions.
2. **Left Asymmetric Difference:** $S_{only_my} = S_{my} \setminus S_{gst}$ -> Invoices in books but absent on GST portal (**MISSING_IN_GST**).
3. **Right Asymmetric Difference:** $S_{only_gst} = S_{gst} \setminus S_{my}$ -> Invoices on portal but omitted in books (**EXTRA_IN_GST**).

3. Classification Taxonomy & Statutory Action Matrix

Every invoice is tagged with a decisive status, a fine-grained discrepancy category, plausible hypotheses, and immediate remediation steps:

Status	Mathematical Condition	Discrepancy Category	Compliance Impact & Recommended Action
MATCH	inv in S_{common} dTV = 0 & dGST = 0	MATCH	Perfect reconciliation across registers. Safe for statutory reporting; zero audit risk.
MISMATCH	inv in S_{common} dTV != 0 or dGST != 0	TAXABLE_AND_GST_MISMATCH TAXABLE_VALUE_MISMATCH GST_AMOUNT_MISMATCH	Data entry typo, rate variance, or unapplied credit note. Compare physical invoices; issue credit/debit note if needed.
MISMATCH	inv in S_{common} $n_{my} > 1$ or $n_{gst} > 1$	DUPLICATE_INVOICE	Duplicate billings detected. Multi-row preserve algorithm retains every entry to prevent silent drop and tax overpayment.
MISSING_IN_GST	inv in $S_{my} \setminus S_{gst}$	MISSING_IN_GST	**Sales:** Unreported outward supply (interest risk). **Purchases:** Vendor has not filed GSTR-1; **ITC is blocked** under Sec 16(2)(aa). Contact vendor.
EXTRA_IN_GST	inv in $S_{gst} \setminus S_{my}$	EXTRA_IN_GST	Unbooked invoice on portal or incorrect recipient GSTIN. Verify delivery challans; enter in purchase journal or reject bill.

4. Function Deep Dive: _extract_gstin(row)

Signature: `_extract_gstin(row) -> str`
Normalizes and extracts the 15-character Goods and Services Taxpayer Identification Number (GSTIN) from diverse data row structures.

```
def _extract_gstin(row) -> str:
    if hasattr(row, "get"):
        return str(row.get("gstin") or row.get("customer_gstin") or row.get("supplier_gstin") or "").strip()
    return ""
```

Detailed Working & Design Mechanics:

- **Polymorphic Type Guarding:** Evaluates hasattr(row, 'get') so the function safely processes Python dictionaries (from JSON/API payloads) and Pandas Series objects identically without raising AttributeError.
- **Short-Circuit Priority Resolution:** Evaluates schema variations in order of frequency: 'gstin' -> 'customer_gstin' (outward registers) -> 'supplier_gstin' (inward registers). The first truthy value is selected.
- **Sanitization & Defensive Strip:** Converts any value into a string and applies .strip() to eradicate trailing spaces, newline symbols, or whitespace artifacts resulting from CSV/Excel parsing.

5. Function Deep Dive: reconcile(my_source, gst_source)

Signature: `reconcile(my_source, gst_source) -> pd.DataFrame`
The primary reconciliation engine. Accepts either in-memory Pandas DataFrames or on-disk file paths, executing bidirectional comparison across eight structured execution phases.

- **Phase 1: Input Ingestion & Isolation:** Checks isinstance(source, pd.DataFrame). If True, invokes .copy() to guarantee functional immutability (ensures caller DataFrames from the API router are never mutated). If False, loads CSV from the project data/ folder using pd.read_csv().
- **Phase 2: Schema Validation & Defensive Imputation:** Checks for mandatory columns: invoice_no, invoice_date, gstin, taxable_value, gst_amount. If any column is absent from user uploads, it is defensively initialized as pd.Series(dtype=object) to prevent downstream runtime KeyError crashes.
- **Phase 3: Identifier Cleansing & Pruning:** Converts invoice numbers to strings (astype(str).str.strip()) to guarantee uniform types. Prunes blank/placeholder rows where invoice_no == '' to eliminate phantom records.
- **Phase 4: Set Theory Partitioning:** Generates unique hash sets my_inv and gst_inv. Computes common = my_inv & gst_inv, only_my = my_inv - gst_inv, and only_gst = gst_inv - my_inv in linear time.
- **Phase 5: 1-to-1 Discrepancy & Tolerance Verification:** For common invoices with exactly one entry on each side (n_my == 1 and n_gst == 1), computes rounded arithmetic differences: tv_diff = round(a_tv - b_tv, 2) and gst_diff = round(a_gst - b_gst, 2). If differences equal zero, assigns MATCH; otherwise assigns MISMATCH with tailored root-cause hypotheses.
- **Phase 6: Safe Duplicate Handling (Fixing the .iloc[0] Data Drop Bug):** If an invoice number appears more than once on either side (n_my > 1 or n_gst > 1), naive implementations call .iloc[0] and silently discard duplicate rows. ReconcilePro iterates up to max(n_my, n_gst), pairing matching rows where available and preserving every duplicate entry as an individual audit row tagged DUPLICATE_INVOICE.
- **Phase 7: Partition Asymmetries (MISSING_IN_GST & EXTRA_IN_GST):** Iterates over only_my and only_gst. For MISSING_IN_GST, generates notifications regarding unfiled sales or blocked vendor ITC. For EXTRA_IN_GST, suggests checking delivery challans or notifying vendors of wrong GSTIN usage.
- **Phase 8: Structured DataFrame Construction:** Aggregates all compiled audit dictionaries into a clean Pandas DataFrame ready for frontend presentation, Excel/CSV export, and ML anomaly scoring.

Zero-Loss Duplicate Preservation Implementation:

```
else:
    # Safe duplicate handling: preserve every row without silent .iloc[0] drop
    max_len = max(n_my, n_gst)
    for idx in range(max_len):
        a = my_matches.iloc[idx] if idx < n_my else None
        b = gst_matches.iloc[idx] if idx < n_gst else None
        a_tv = float(a["taxable_value"]) if a is not None else None
        b_tv = float(b["taxable_value"]) if b is not None else None
        tv_diff = round(a_tv - b_tv, 2) if (a_tv is not None and b_tv is not None) else None
        explanation = f"Duplicate invoice number '{inv}' detected ({n_my} in books, {n_gst} in GST)."
        # Appends all individual rows to results list ensuring complete audit visibility
```

6. Function Deep Dive: show_result & CLI Entry Point

Signature: `show_result(title, df) -> None`
Outputs terminal formatting, displays tabular results without index numbers (to_string(index=False)), and prints a frequency distribution of reconciliation statuses via df['status'].value_counts().

```
def show_result(title, df):
    print("\n" + "=" * 60)
    print(title)
    print("=" * 60)
    print(df.to_string(index=False))
    print("\nSummary:")
    print(df["status"].value_counts())

if __name__ == "__main__":
    sales_result = reconcile("sales_register.csv", "gstr1.csv")
    purchase_result = reconcile("purchase_register.csv", "gstr2a.csv")
    show_result("SALES RECONCILIATION: SALES REGISTER vs GSTR-1", sales_result)
    show_result("PURCHASE RECONCILIATION: PURCHASE REGISTER vs GSTR-2A", purchase_result)
```

7. End-to-End System Integration Architecture

Within the ReconcilePro production stack, reconcile.py is cleanly decoupled from storage, serving as an in-memory computational engine:

- **Step 1: REST Request:** The React client initiates a call to GET /reconcile?tax_period=YYYY-MM&purchase;_source=gstr2a with Bearer JWT credentials.
- **Step 2: Tenant Isolation & Ingestion:** FastAPI's dependency injector authenticates the business tenant, queries PostgreSQL/SQLite for tenant invoices in that period, and constructs in-memory DataFrames for both registers.
- **Step 3: Core Engine Execution:** The route handler executes reconcile(my_df, gst_df). The engine completes set partitioning and variance calculation in milliseconds.
- **Step 4: Machine Learning Anomaly Detection:** The resulting audit DataFrame is passed to services/ml_service.py, which runs an Isolation Forest and rule-based risk model, generating normalized Risk Scores (0-100) and tagging suspicious tax ratios.
- **Step 5: Client Dashboard & Statutory Reports:** The API returns the unified dataset to the React frontend for interactive filtering, KPI cards, and statutory PDF/Excel exports.

8. Complete Output Schema Reference Table

The DataFrame returned by reconcile() provides 16 standardized audit columns, powering both frontend UI views and statutory audit exports:

Column Name	Data Type	Sample / Domain	Functional Role & Business Meaning
invoice_no	String	INV-2026-001	Cleaned alphanumeric invoice number; primary matching key.
invoice_date	String (YYYY-MM-DD)	2026-07-15	Invoice date from books or portal filing.
gstin	String (15-char)	29AAAAA0000A1Z5	Counterparty customer/supplier GSTIN resolved via _extract_gstin.
taxable_value	Float	100000.00	Display taxable value (defaults to books; falls back to portal).
gst_amount	Float	18000.00	Display GST amount (defaults to books; falls back to portal).
expected_taxable_value	Float / None	100000.00	Taxable value according to internal ledger.
actual_taxable_value	Float / None	95000.00	Taxable value according to filed GST portal return.
expected_gst_amount	Float / None	18000.00	Total GST (CGST + SGST + IGST) according to internal ledger.
actual_gst_amount	Float / None	17100.00	Total GST according to filed GST portal return.
status	Enum String	MATCH / MISMATCH	High-level status: MATCH, MISMATCH, MISSING_IN_GST, EXTRA_IN_GST.
taxable_value_diff	Float / None	5000.00	Arithmetic difference: round(expected_tv - actual_tv, 2).
gst_amount_diff	Float / None	900.00	Arithmetic difference: round(expected_gst - actual_gst, 2).
explanation	String	Audit sentence	Human-readable statement detailing exact rupee variances.
discrepancy_category	Enum String	TAXABLE_AND_GST...	Fine-grained taxonomy category for automated reporting & filtering.
possible_causes	List[String]	Hypotheses list	Root-cause hypotheses (e.g. rate mismatch, credit note omission).
suggested_action	String	Action item	Immediate corrective action for the finance/accounting team.